

Theory Of Computation

FINITE AUTOMATA

CHOMSKY'S CLASSIFICATION

Regular Expression Entities :-

$$\Phi R + R = R$$

$$\Phi R + R \Phi = \Phi$$

$$\epsilon R = R \epsilon = R$$

$$\epsilon^* = \epsilon \text{ \& } \Phi^* = \Phi \epsilon$$

$$R + R = R$$

$$RR^* = R^*R$$

$$(R^*)^* = R^*$$

$$\epsilon + RR^* = R^*$$

$$(PQ)^*P = P(QP)^*$$

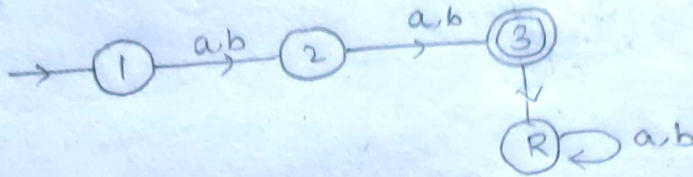
$$(P+Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$$

$$(P+Q)R = PR + QR$$

L1: All strings length exactly 2, $\Sigma = \{a, b\}$.

$$L = \{ "aa", "bb", "ab", "ba" \}$$

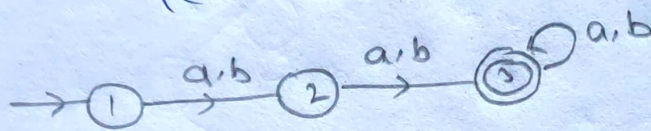
DFA



L2: All string of length atleast 2, $\Sigma = \{a, b\}$.

$$L = \{ 'ab', 'aa', 'bb', 'ba', 'aba', \dots \}$$

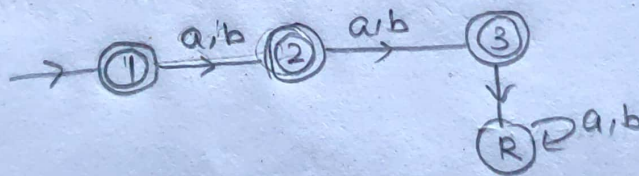
$$= ((a+b)^2 + (a+b)^*)$$



L3: All string of length atmost 2, $\Sigma = \{a, b\}$

$$L = \{ \epsilon, 'a', 'b', 'ab', 'ba', 'aa', 'bb' \}$$

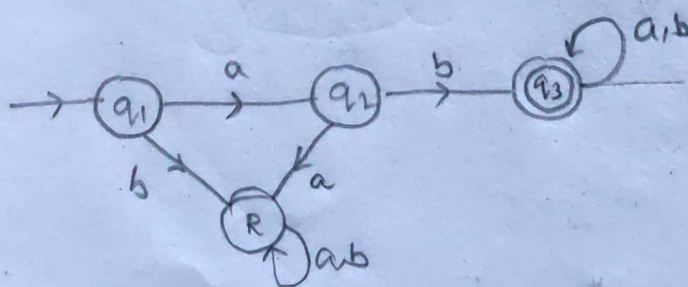
$$= \{ (a+b)^* \} \quad ?$$



L4: All strings starting with 'ab', $\Sigma = \{a, b\}$

$$L = \{ ab, aba, abb, abaa, \dots \}$$

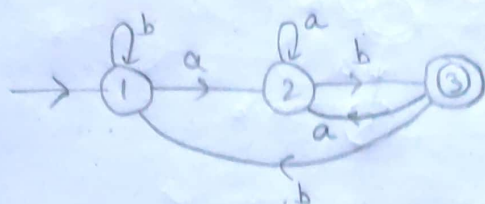
$$= \{ ab + (a+b)^* \}$$



L5: All strings ending with 'ab', $\Sigma = \{a, b\}$.

$L = \{ 'ab', 'aab', 'bab', 'abab' \dots \}$

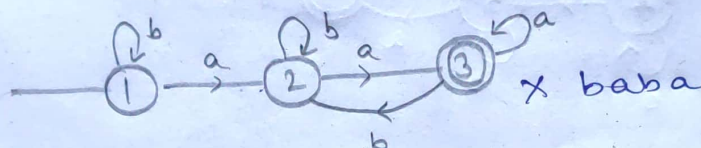
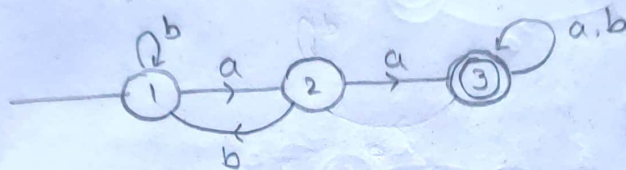
$$= \{ (a+b)^* + ab \}$$



L6: All strings having 'aa' as a substring, $\Sigma = \{a, b\}$

$L = \{ aa, aaa, baaa, abaa, \dots \}$

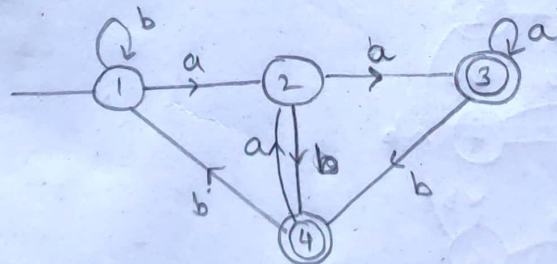
$$= \{ (a+b)^* aa (a+b)^* \}$$



L7: All string where 2nd last character will be 'a'.

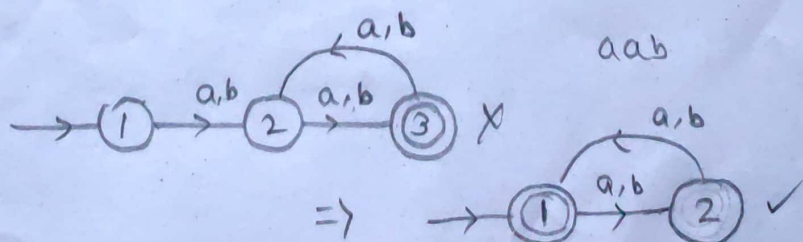
$L = \{ 'aa', 'ab', 'bab', 'aaa' \dots \}$

$$= \{ (a+b)^* a (a+b)^+ \}$$

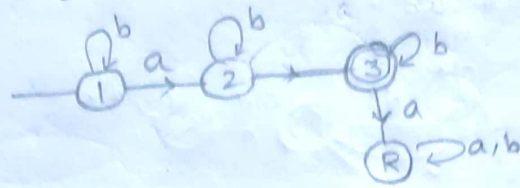


L8: All strings where length is divisible by 2.

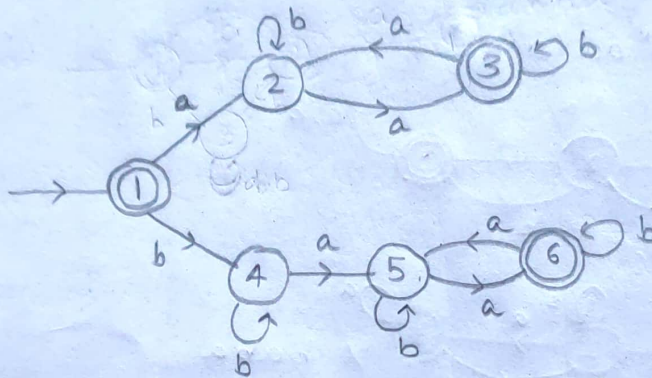
$$L = \{ (a+b)^{2n} \mid n = \{0, 1, 2, 3, \dots\} \}$$



L9: All strings where no. of a is exactly 2. $\Sigma = \{a, b\}$
 $L = \{ \text{ababab}, 'aa', 'aba', \text{ababab}, \dots \}$

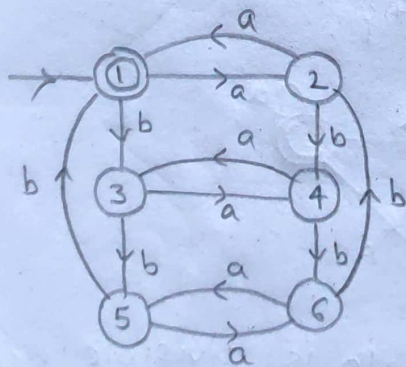


L10: All string where ^{count}length of 'a' is div by 2.



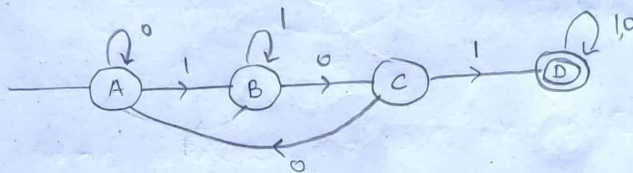
Learn

L11: All strings where length of 'a' is divisible by 2 and length of b div by 3.

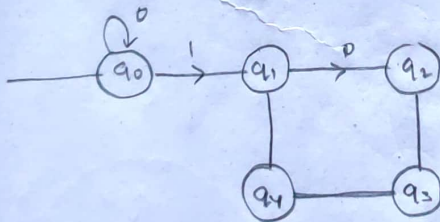


L12: All strings where length of 'a' is atleast 3 and length of 'b' atmost 4. $\Sigma = \{a, b\}$.

L13: All strings where '101' is a substring, $\Sigma = \{0, 1\}$.
 $L = \{101, 1101, 11010, \dots\}$
 $= \{(1+0)^*101(101)^*\}$



* L14: All strings whose decimal converted value will be divisible by 5, $\Sigma = \{0, 1\}$



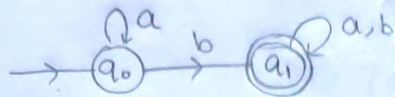
div by n .

	0	1
q_0	q_0	q_1
q_1	q_2	q_3
$\vdots$	$\vdots$	$\vdots$
q_n		q_n

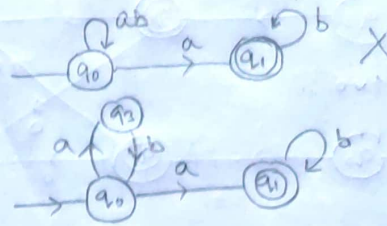
state	0	1
q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_4	q_5
q_3	q_0	q_1
q_4	q_2	q_3

REGULAR EXPRESSION TO NFA

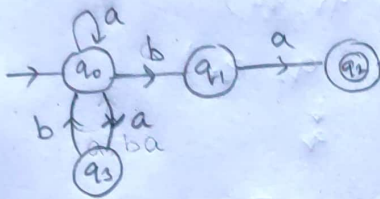
1) $a^*b(a+b)^*$



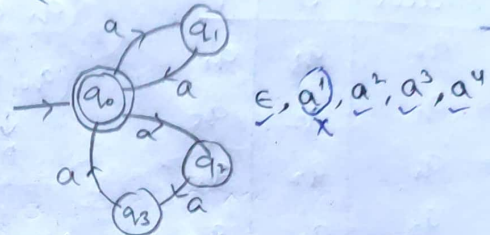
2) $(ab)^*ab^*$



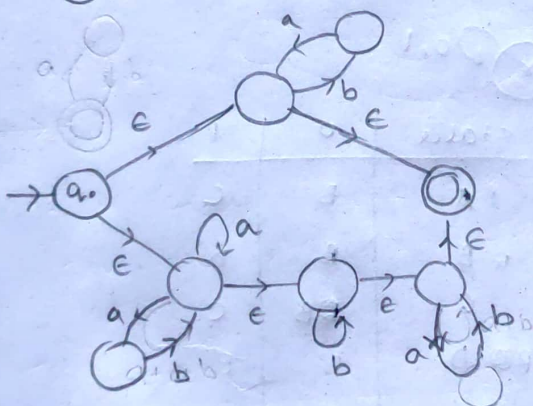
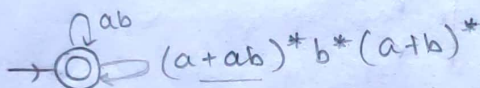
3) $(a+ba)^*ba$



4) $(aa+aaa)^*$



5) $(ab)^* + (a+ab)^*b^*(a+b)^*$

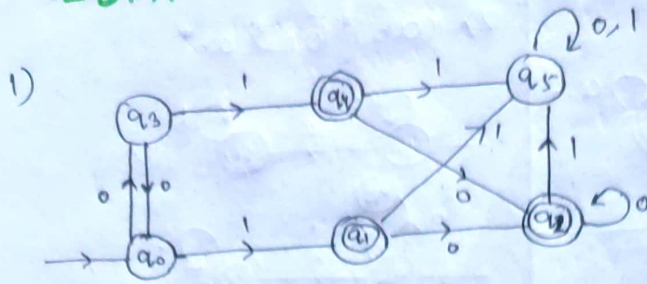


just 1 state is required.

6) $[a+ba(ab)^*]^*a(ba)^*b^*$



DFA MINIMISATION



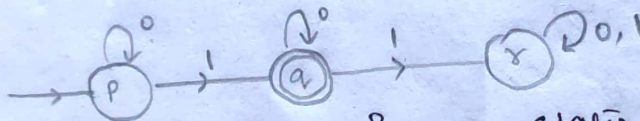
state transition table.

states	0	1
q ₀	q ₁	q ₃
* q ₁	q ₂	q ₅
* q ₂	q ₂	q ₅
q ₃	q ₀	q ₄
* q ₄	q ₂	q ₅
q ₅	q ₅	q ₅

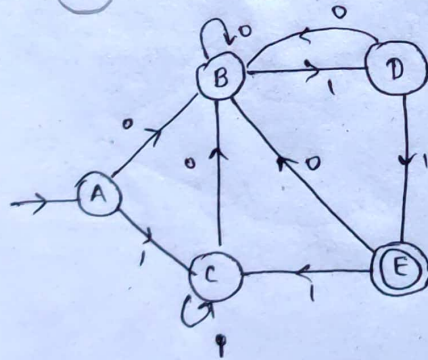
(q_0, q_3, q_5) (q_1, q_2, q_4)
 (q_0, q_3) (q_3) (q_1, q_2, q_4)
 $\downarrow$ $\downarrow$ $\downarrow$
 p r q

New transition state.

states	0	1
p	p	q
* q	q	r
r	r	r



2)



states	0	1
p A	B	C
q B	B	D
r C	B	C
* s D	B	E
t E	B	C

$(ABCD)$ (E)

(ABC) (D) (E)

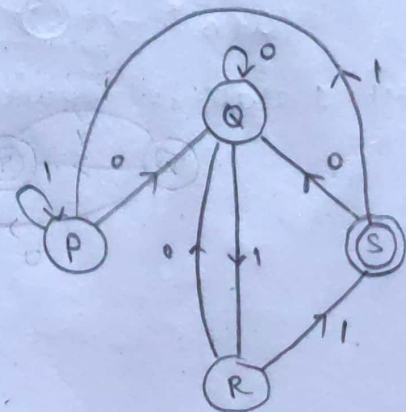


states	0	1
p	p	p
q	p	r
r	p	p

(Ac) (B) (D) (E)

$\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
 p q r s

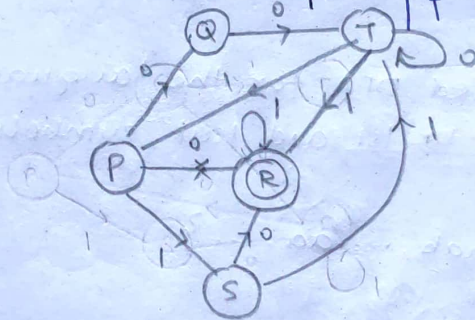
states	0	1
p	q	p
q	q	r
r	q	s
* s	q	p



3) Construct a minimum DFA equivalent to the DFA describe by.

states	0	1	
$\rightarrow q_0$	q_1	q_5	$\Rightarrow (q_0, q_1, q_3, q_4, q_5, q_6, q_7) (q_2)$
q_1	q_6	q_2	$\Rightarrow (q_0, q_4, q_6) (q_1, q_7) (q_2)$
$* q_2$	q_0	q_2	(q_3, q_5)
q_3	q_2	q_6	$\Rightarrow (q_0, q_4) (q_6) (q_1, q_7) (q_3, q_5) (q_2)$
q_4	q_7	q_5	
q_5	q_2	q_6	
q_6	q_6	q_4	
q_7	q_6	q_2	

states	0	1
P	Q	S
Q	T	R
* R	P	R
S	R	T
T	T	P

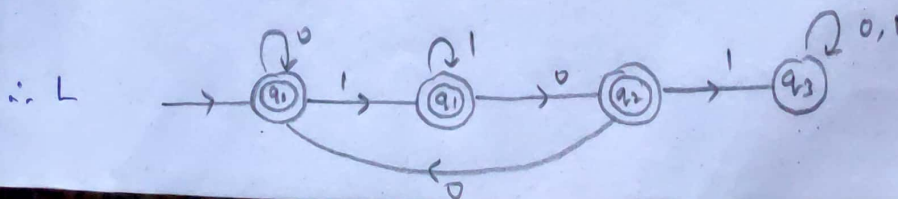
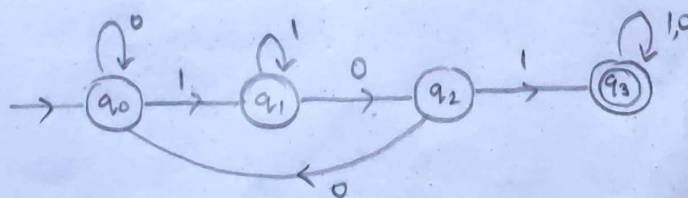


4) Construct a minimised DFA for language
 $L = \text{strings not having '01' substring } \Sigma = \{0, 1\}.$

$L^c = \text{string having '01' as substring } \Sigma = \{0, 1\}.$

$$L = \{101, 01011, \dots\}$$

$$= \{(1+0)^* 101 (1+0)^*\}$$

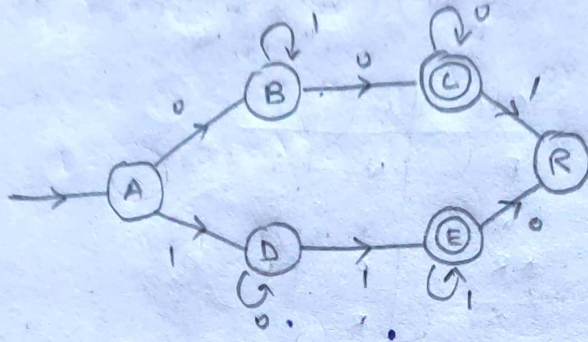


5) Construct a minimised DFA for the language
 $L = \{ \text{strings where last two symbols are same} \}$
 over alphabet $\Sigma = \{0, 1\}$.

~~$L = \{0110, 10101, \dots\}$~~

~~$L = (0+1)^* = \{00, 11, 000, 1000, \dots\}$~~

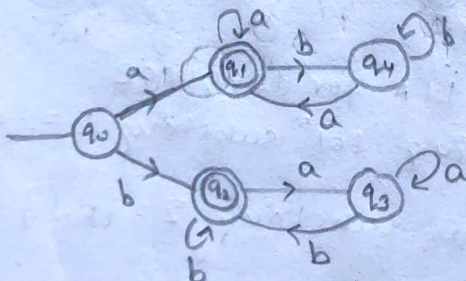
$L = (0+1)^*00 + (0+1)^*11$



6) Construct a minimised DFA for the language.

$L = \{ \text{string where 1st and last symbols are same} \}$
 $\Sigma = \{a, b\}$

$L = \{ abba, baab, \dots, aa, \dots \}$



05/11/23

NFA : Non Deterministic Finite Automata.

$(Q, \Sigma, q_0, F, \delta)$

Q = set of all states.

Σ = inputs.

q_0 = start state / initial state.

F = set of Final states.

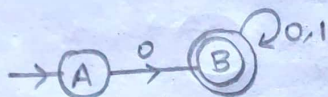
$\delta = Q \times \Sigma \rightarrow 2^Q$ (2^Q)

→ dead configuration.
(Reject state).

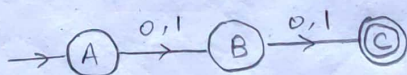
1) Construct a NFA for the language:-

$L = \forall$ strings starting with '0' $\Sigma = \{0, 1\}$

$L = \{0, 00, 001, 01, \dots\}$

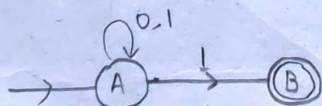


2) $L = \forall$ strings accepting self of length 2. $\Sigma = \{0, 1\}$
 $L = \{00, 01, 10, 11\}$



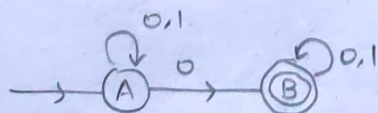
3) $L =$ ending with '1'.

$L = \{1, 01, 11, 0001, \dots\}$



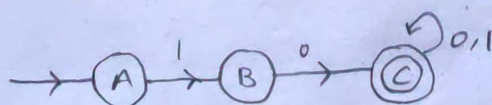
4) $L =$ set of all strings that contain 0.

$L = \{010, 0, 1001, \dots\}$



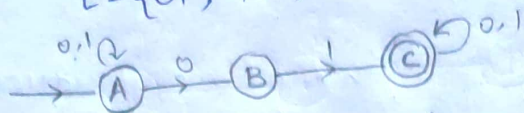
5) $L =$ starting with '10'

$L = \{10, 1000, 1011, \dots\}$



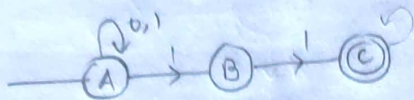
6) $L = \text{contain '01'}$.

$$L = \{01, 101, 0101, \dots\}$$



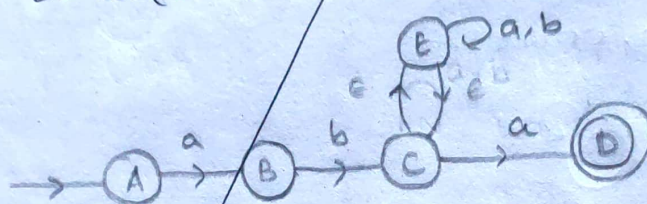
7) $L = \{\text{ending with '11'}\}$

$$L = \{11, 011, 0011, 1111, \dots\}$$



Q) Construct a ϵ -NFA of the language.

$$L = ab(a+b)^*a \quad \Sigma = \{a, b\}$$



ϵ -NFA

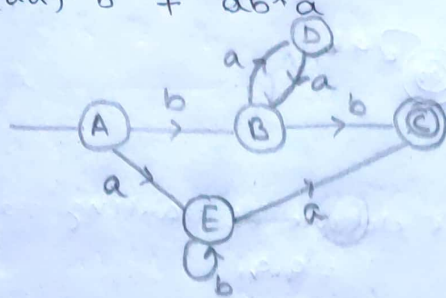
Convert ϵ NFA to DFA.

State	0	1
$\rightarrow A$		
B		
C		
D		
E		

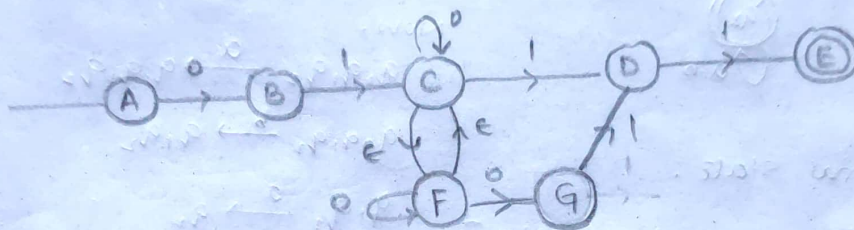
next page

Construct NFA / ENFA for RE...

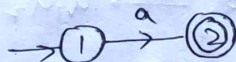
1) $b(aa)^*b + ab^*a$



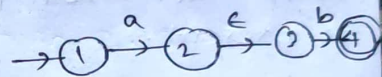
2) $01[(0+011^*)^* + 0]^*11$



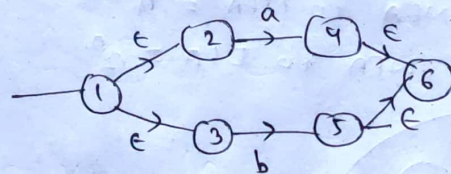
(1) a



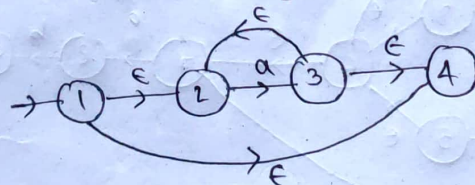
(2) ab



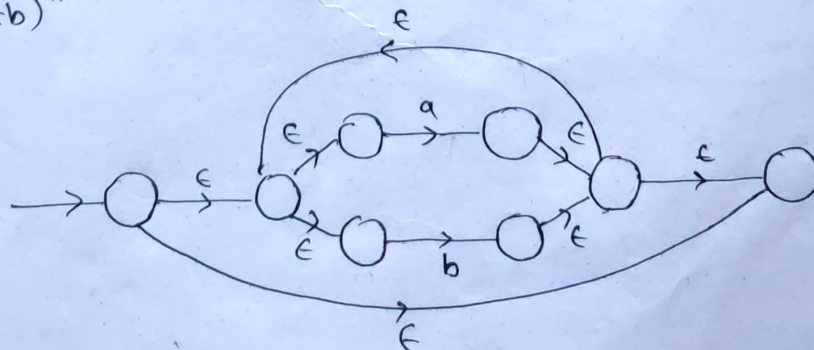
(3) $a+b$



(4) a^*



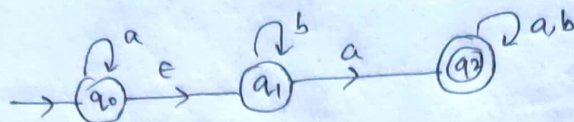
(5) $(a+b)^*$



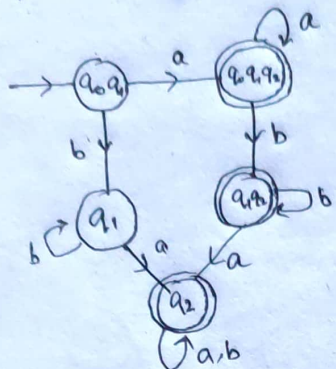
16/11/23

CONVERSION OF ENFA TO DFA

①



step 1: initial state.



ϵ closure of starting state

$$= q_0, q_1$$

$$q_0, q_1 \xrightarrow{a} q_0, q_1, q_2$$

$$q_0, q_1 \xrightarrow{b} q_1$$

Now,

$$q_0, q_1, q_2 \xrightarrow{a} q_0, q_1, q_2$$

$$q_0, q_1, q_2 \xrightarrow{b} q_1, q_2$$

$$q_1, q_2 \xrightarrow{a} q_2$$

$$q_1, q_2 \xrightarrow{b} q_1, q_2$$

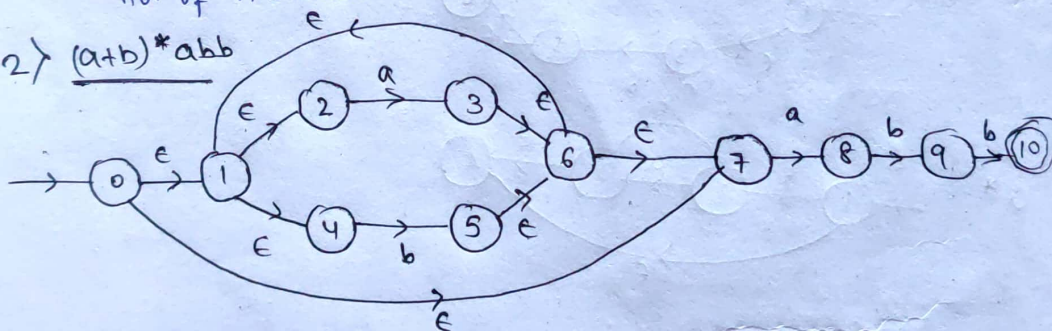
Step 2: Final state:

every states where final state is present.

Note: no. of states in ENFA to DFA = n

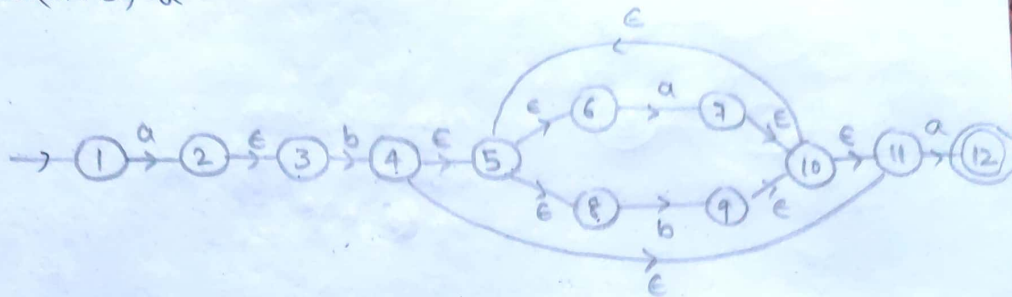
no. of n states in ENFA to DFA = 2^n

2) $(a+b)^*abb$



CONSTRUCT ENFA for

① $ab(a+b)^*a$



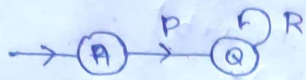
Now convert it into DFA :

state transition table.

states	a	b
1 $\textcircled{P}$	2 B	R
2 B	3	$\{3, 4, 5, 6, 8, 9, 10, 11\}$
C $\{3, 4, 5, 6, 7, 8, 9, 10, 11\}$	$\{5, 6, 7, 8, 10, 11, 12\}$	$\{4, 5, 6, 8, 9, 10, 11\}$
D $\{5, 6, 7, 8, 10, 11, 12\}$	$\{5, 6, 7, 8, 10, 11, 12\}$	$\{6, 8, 9, 10, 11\}$
E $\{4, 5, 6, 8, 9, 10, 11\}$	$\{5, 6, 7, 8, 10, 11, 12\}$	$\{5, 6, 8, 9, 10, 11\}$

ARDEN'S THEOREM

USED TO FIND REGULAR EXPRESSION :-



$$\begin{aligned} \text{iff } Q &= P + QR \\ \therefore Q &= PR^* \end{aligned}$$

$$Q = P + QR$$

$$= P + (P + QR)R$$

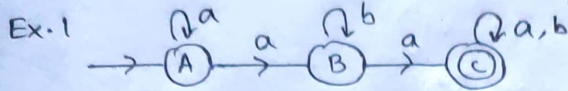
$$= P + PR + QR^2$$

$$= P + PR + (P + QR)R^2$$

$$= P + PR + PR^2 + QR^3$$

$$= P(1 + R + R^2 + \dots)$$

$$\therefore PR^*$$



$$A = \epsilon + aA = a^*$$

$$B = Aa + Bb = Aab^* = a^*ab^* = a^+b^*$$

$$C = aB + C(a+b) = aB(a+b) = a(a^+b^*)(a+b)$$

$$\boxed{C = (a^+b^*)(a+b)}$$

Q) Prove that $(1+00^*1) + (1+00^*1)(0+10^*1)^*(0+10^*1) = 0^*1(0+10^*1)^*$

LHS :-

$$= (1+00^*1) + (1+00^*1)(0+10^*1)^*(0+10^*1)$$

$$= (1+00^*1) [\epsilon + (0+10^*1)^*(0+10^*1)] \quad \boxed{\epsilon + R^*R = R^*}$$

$$= (1+00^*1) [\epsilon + (0+10^*1)^* (0+10^*1)]$$

$$= (1+00^*1)(0+10^*1)^*$$

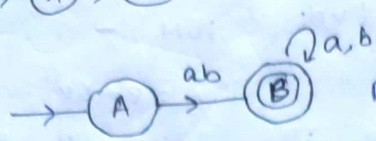
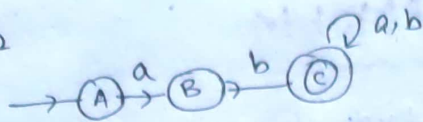
$$= (\epsilon + 00^*)1(0+10^*1)^*$$

$$= 0^*1(0+10^*1)^*$$

$$= \text{RHS hence proved}$$

$$\boxed{\epsilon \cdot R = R}$$

Eg-2

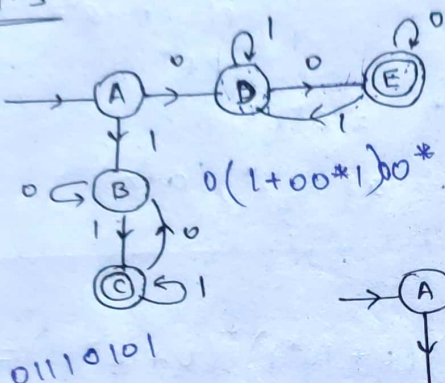


$$B = Aab + B(a+b)$$

$$= \epsilon ab + B(a+b)$$

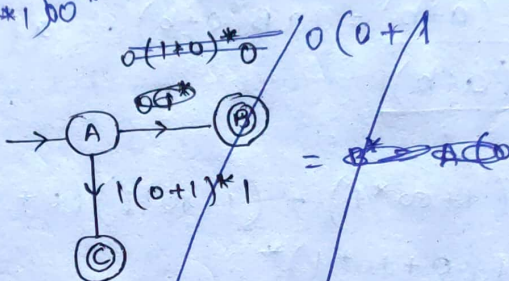
$$= ab(a+b)^*$$

Eg-3



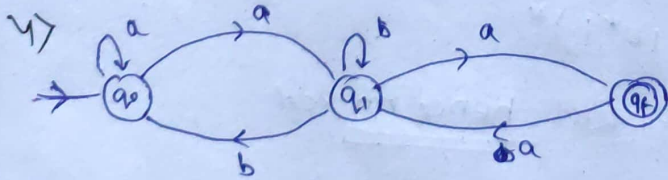
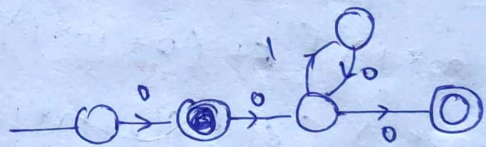
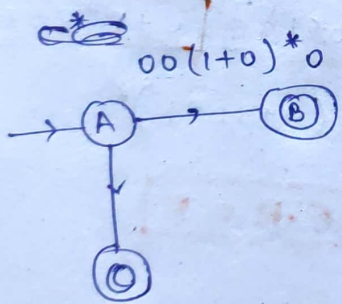
$$01^*0$$

$$01^*01^*$$



$$B^* = 0(1+0)^*0A + \epsilon$$

$$= 0(1+0)^*0$$



$$q_0 = q_0a + q_1b + \epsilon$$

$$q_1 = q_0a + q_1b + q_fa$$

$$q_f = q_1a$$

$$q_0a + q_1b + q_1a^2$$

$$q_1 = q_0a + q_1(b+a)$$

$$= q_0a(b+a)^*$$

$$q_0 = \epsilon + q_1 b + q_0 a$$

$$= \epsilon + q_0 a^2 (b + a^2)^* + q_0 a$$

$$= \epsilon + q_0 (a^2 (b + a^2)^* + a)$$

$$q_0 = [a^2 (b + a^2)^* + a]^*$$

$$q_0 = \epsilon + q_1 b + q_0 a$$

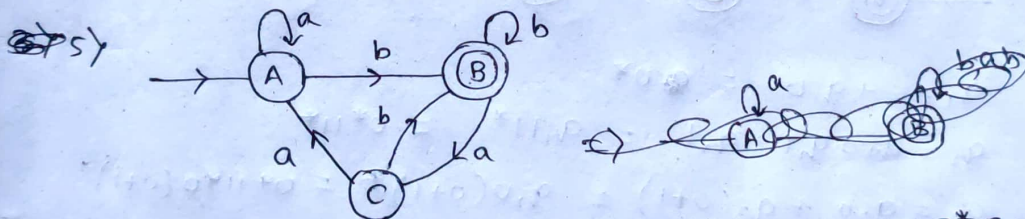
$$= \epsilon + [q_0 a (b + a^2)^*] b + q_0 a$$

$$= \epsilon + q_0 [a (b + a^2)^* b + a]$$

$$= [a + a (b + a^2)^* b]^*$$

$$\cancel{q_1} \quad q_1 = q_0 a (b + a^2)^* = [a + a (b + a^2)^* b] a (b + a^2)^*$$

$$q_1 = [a + a (b + a^2)^* b] a (b + a^2)^* a$$



$$\begin{aligned} \epsilon + R^* R &= R^* \\ \epsilon R &= R \end{aligned}$$

$$A = \epsilon + aA + aC \Rightarrow aCa^* \Rightarrow \underline{a a B a^*}$$

$$B = bA + bB + bC \Rightarrow B = bA + bB + aBb$$

$$C = aB$$

$$\begin{aligned} &= bA + B(b + ab) \\ &= bA(b + ab)^* \end{aligned}$$

$$B = b(a a B a^*)(b + ab)^*$$

$$B = b(\epsilon + a a B a^*)(b + ab)^*$$

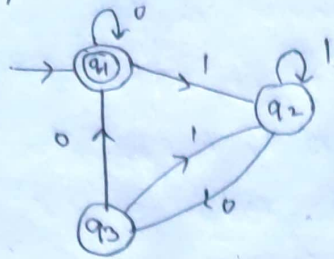
$$B = b(\epsilon + B a a a^*)(b + ab)^*$$

$$B = b(a a a^* b + b + ab)^*$$

$$b a a^* | + | + |$$

next page

57



$$q_1 = \epsilon + q_1 0 + q_3 0$$

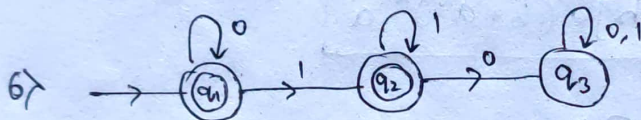
$$q_2 = q_1 1 + q_2 1 + q_3 1$$

$$q_3 = q_2 0$$

$$\begin{aligned} q_2 &= q_1 1 + q_2 1 + q_2 0 1 \\ &= q_1 1 + q_2 (1 + 01) \\ &= q_1 1 (1 + 01)^* \end{aligned}$$

$$q_3 = q_1 1 (1 + 01)^* 0$$

$$\begin{aligned} \therefore q_1 &= q_1 0 + q_1 1 (1 + 01)^* 00 + \epsilon \\ &= \epsilon + q_1 (0 + 1 (1 + 01)^* 00) \\ &= (0 + 1 (1 + 01)^* 00)^* \quad (\text{ans}) \end{aligned}$$



$$q_1 = \epsilon + q_1 0 = 0^*$$

$$q_2 = q_1 1 + q_2 1 = q_1 1^* = 0^* 1^*$$

$$q_3 = q_2 0 + q_3 (0 + 1) = q_2 0 (0 + 1)^* = 0^* 1^* 0 (0 + 1)^*$$

$$\therefore RE : 0^* + (0^* 1^*)$$

PUMPING LEMMA

For Regular language.

If any language is Regular then:

- there will be pumping length p and a string w , where $w \in L$ and we can divide $w = xyz$ such that

i) $|xy| \leq p$

ii) $|y| > 1$

iii) for $\forall i, xy^i z \in L$

Ex-1

$L = \{a^n b^n \mid n > 1, \Sigma = \{a, b\}\}$ is not Regular lang

Suppose L is a regular lang.

consider $w = aabb$ + let $p = 3$

$$x = a \quad y = ab \quad z = b$$

i) $|xy| \leq p \quad aab = 3 \leq 3 \quad \checkmark$

ii) $|y| > 1 \quad ab = 2 > 1 \quad \checkmark$

iii) $xy^i z \in L$

let $i = 3$

$$xy^3 z \Rightarrow a(ab)^3 b = aabababb \notin L$$

Hence prove this is not regular expression.

Ex-2

$L = \{a^p \mid p \text{ is a prime no.}, \Sigma = \{a\}\}$

consider $w = aaaaa$

let $p = 5$

$$x = a \quad y = aaa \quad z = a$$

i) $|xy| \leq p \quad aaaa \leq 5 \quad \checkmark$

ii) $|y| > 1 \quad y = aaa = 3 > 1 \quad \checkmark$

iii) $xy^i z$ let $i = 2$

$$xy^2 z = a(aaa)^2 a = aaaaaaaa = a^8 \notin L$$

Ex-3

using Pumping lemma prove that the language

$A = \{yy \mid y \in \{0,1\}^*\}$ is not regular.

suppose A is a regular language.

let $w = 010010$ $y = 010 \Rightarrow w = 010010$

$x = 010$ $y = 010$ $z = 0$ let $p = 6$

i) $|xy| \leq p$ $010010 = 5 \leq 6 \checkmark$

ii) $|y| > 1$ $010 > 1 \checkmark$

iii) xy^iz let $i = 2$

$xy^iz = 0100100100$ $010(01)^20$
 $= 01001010 \notin A$

Hence A is not a regular language.

MEALY MOORE MACHINE

- 1) Construct a moore machine that prints 'a' whenever the sequence '01' is encountered in any input binary string.

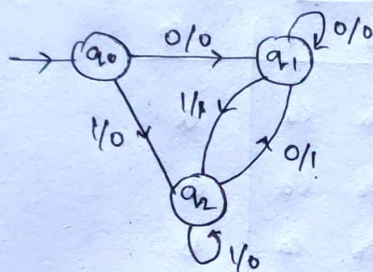
STEP 1 : Draw the DFA.

STEP 2 : No final state just output. $Z = \{0, 1\}$



Mealy to Moore Machine

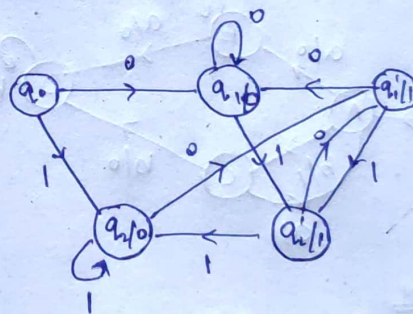
transition table



state	NS	O/P	NS	O/P
→ q ₀	q ₁	0	q ₂	0
q ₁	q ₁	0	q ₂	1
q ₂	q ₁	1	q ₂	0

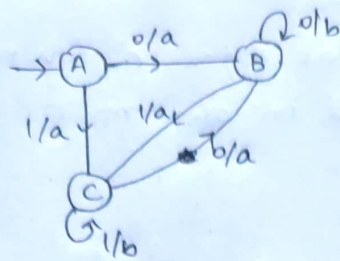
moore transition table

states	NS		O/P
	0	1	
→ q ₀	q ₁₀	q ₂₀	0
q ₁₀	q ₁₀	q ₂₁	0
q ₁₁	q ₁₀	q ₂₁	1
q ₂₀	q ₁₁	q ₂₀	0
q ₂₁	q ₁₁	q ₂₀	1

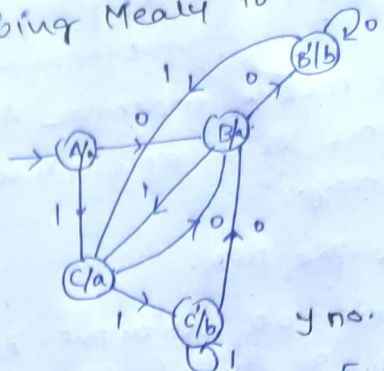


no. of states = 5

2) Convert the following Mealy to Moore Machine



x states



y no. states.

no. of states = 5

∴ Max (xxy) no of states.

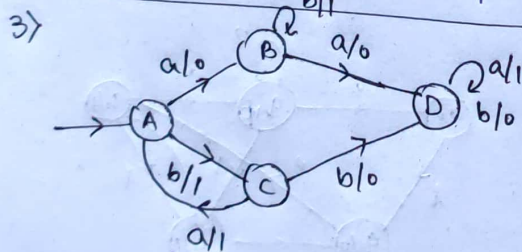
Now using transition table.

Mealy

states	NS	0/p	NS	OP
→ A	B	a	C	a
B	B	b	C	a
C	B	a	C	b

Moore

states	0	1	O/P
→ A	Ba	Ca	a
Ba	Bb	Ca	a
Bb	Bb	Ca	b
Ca	Ba	Cb	a
Cb	Ba	Cb	b



Mealy

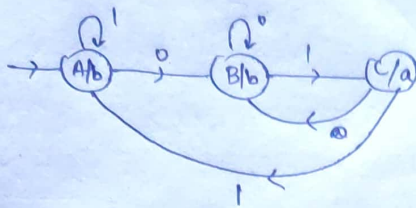
states	a		b	
	NS	OP	NS	OP
→ A	B	0	C	1
B	D	0	B	1
C	A	1	D	0
D	D	1	D	0

Moore

states	a	b	O/P
→ A	B ₀	C	1
B ₀	D ₀	B ₁	0
B ₁	D ₀	B ₁	1
C	A	D ₀	1
D ₀	B ₁	D ₀	0
D ₁	D ₁	D ₀	1

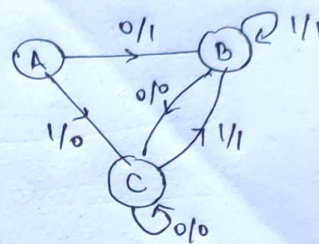
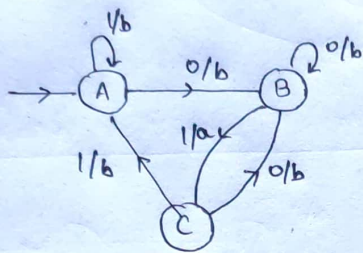
Convert Moore to Mealy Machine.

1) $\Sigma = \{0, 1\}$ $\delta = \{a, b\}$



state	0	1	o/p
→ A	B, b	A, b	b
B	B, b	C, a	b
C	B, b	A, b	a

Mealy



2)

state	0	1	o/p
→ A	B	C	0
B	C	B	1
C	C	B	0

=>

↑

state	0	1
→ A	B ₀	C ₀
B	C ₀	B ₁
C	C ₀	B ₁